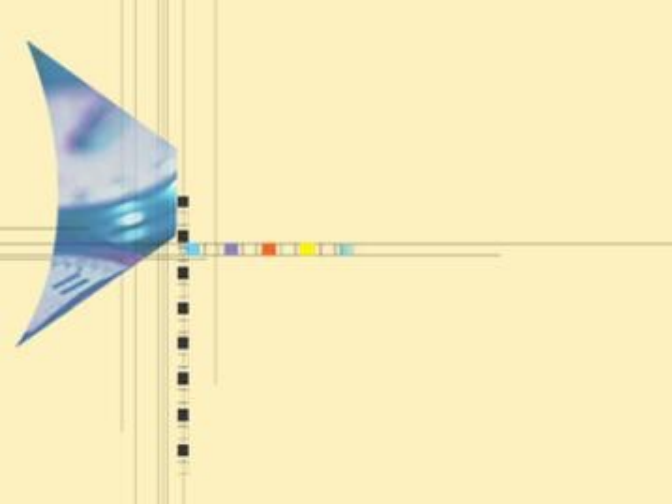




算法设计与分析

主讲：刘娟，武汉大学人工智能学院

2024-2025 第二学期，1-16周

周一(3-4，三区1-502)，周三（6-7，三区1-325）

第三讲

分 治



主要内容



3.1 引言

3.2 分治算法的简单例

3.3 分治策略的思想

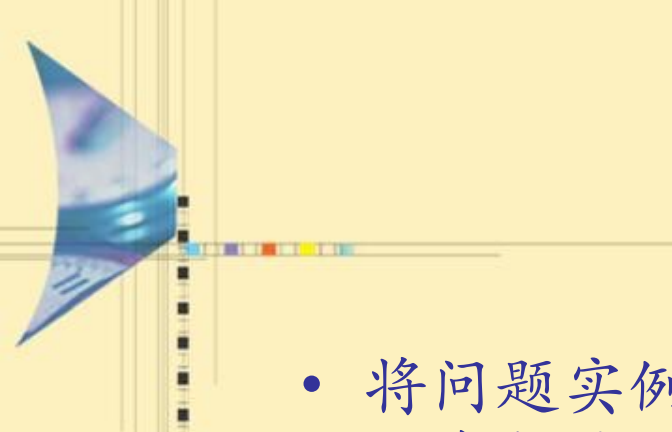
3.4 分治策略的经典应用

3.5 分治算法的改进



分治法

- 对于一个规模为 n 的问题：若该问题可以容易地解决（比如说规模 n 较小）则直接解决，否则将其分解为 p 个规模较小的子问题，这些子问题互相独立且与原问题形式相同，递归地解这些子问题，然后将各子问题的解合并得到原问题的解。
- 这种算法设计策略叫做分治法。

- 
- 将问题实例划分成若干个问题子实例（子问题），大多数情况是分成2个子实例；
 - 对每个子实例，分别递归求解，得到子解；
 - 组合子实例的解，得到原问题实例的解。

分治法和归纳法

- 分治法与归纳法像一对孪生兄弟，经常同时应用在算法设计之中，并由此产生许多高效算法。
- 由分治法/归纳法产生的子问题往往是原问题的较小模式，这就为使用递归技术提供了方便。
- 两者都是反复从原问题得到子问题，使子问题与原问题类型一致而其规模却不断缩小，最终使子问题缩小到很容易直接求出其解。自然导致递归过程的产生
- 不同点在于：归纳法是尾递归思想，通过扩展子问题解的方式得到原问题的解；而分治法是划分的思想，通过合并多个子问题的解决得到原问题的解。

例：寻找最大和最小元素问题

问题描述： 在一个整数数组 $A[1..n]$ ，同时寻找最大和最小元素。
为简单期间，不妨假定 n 是2的幂。

直接求解算法如下：

```
1.  $x \leftarrow A[1]; \quad y \leftarrow A[1]$   
2. for  $i \leftarrow 2$  to  $n$   
3.   if  $A[i] < x$  then  $x \leftarrow A[i]$   
4.   if  $A[i] > y$  then  $y \leftarrow A[i]$   
5. end for  
6. return  $(x, y)$ 
```

元素比较总次数
为 $2n-2$.

Much more efficient method?

利用分治策略的方法

- 基本思想

- 将数组 $A[1..n]$ 分成2个子数组 $A[1..n/2]$, $A[(n/2)+1..n]$;
- 在每个子数组中, 分别递归寻找各自的最大和最小元素;
- 返回将两个子数组的最小元素中更小的, 两个子数组的最大元素中更大的作为整个数组的最大和最小元素。

Algorithm 3.1 MINMAX

Input: n 个整数元素的数组 $A[1..n]$, n 是 2 的幂.

Output: (x,y) : A 中的最小元素 x 和最大元素 y .

1. $\text{minmax}(1,n)$

Procedure $\text{minmax}(\text{low}, \text{high})$

1. **if** $\text{high} - \text{low} = 1$ **then**

2. **if** $A[\text{low}] < A[\text{high}]$ **then return** $(A[\text{low}], A[\text{high}])$

3. **else return** $(A[\text{high}], A[\text{low}])$

4. **end if**

5. **else**

6. $\text{mid} \leftarrow \lfloor (\text{low} + \text{high}) / 2 \rfloor$

7. $(x_1, y_1) \leftarrow \text{minmax}(\text{low}, \text{mid})$

8. $(x_2, y_2) \leftarrow \text{minmax}(\text{mid} + 1, \text{high})$

9. $x \leftarrow \min(x_1, x_2)$

10. $y \leftarrow \max(y_1, y_2)$

11. **return** (x, y)

12. **end if**

算法分析

设 $C(n)$ 表示元素比较总次数, n 是2的幂, 则:

$$C(n) = \begin{cases} 1 & \text{if } n = 2 \\ 2C(n/2) + 2 & \text{if } n > 2 \end{cases}$$

定理2.1 设数组 $A[1..n]$ 包含 n 个元素, n 是2的幂, 仅使用 $(3n/2)-2$ 次元素比较在数组 A 中找出最大和最小元素是可能的.

主要内容



3.1 引言

3.2 分治算法的简单例

3.3 分治策略的思想

3.4 分治策略的经典应用

3.5 分治算法的改进



3.1.1 二分搜索

问题描述

- 在一个已按升序排序的数组 $A[1..n]$ 中搜索给定的元素 x

基本思想

将一个给定的元素 x 与一个已排序数组 $A[\text{low}..\text{high}]$ 的中间元素 $A[\text{mid}]$ 做比较，这里 $\text{mid} = \lfloor (\text{low} + \text{high})/2 \rfloor$

- 如果 $x < A[\text{mid}]$ ，则不考虑 $A[\text{mid}..\text{high}]$ ，而对 $A[\text{low}..\text{mid}-1]$ 重复实施相同的方法。
- 类似地，如果 $x > A[\text{mid}]$ ，则放弃 $A[\text{low}..\text{mid}]$ ，而对 $A[\text{mid}+1..\text{high}]$ 重复实施相同的方法。

二分搜索算法

算法3.2 BINARYSEARCHREC

输入: 按非降序排列的 n 个元素的数组 $A[1..n]$ 和元素 x 。

输出: 如果 $x=A[j]$, $1 \leq j \leq n$, 则输出 j ; 否则输出0

1. *binarysearch*(1, n)

过程 *binarysearch*($low, high$)

1. **if** $low > high$ **then return** 0

2. **else**

3. $mid \leftarrow \lfloor (low + high) / 2 \rfloor$

4. **if** $x = A[mid]$ **then return** mid

5. **else if** $x < A[mid]$ **then return** *binarysearch*($low, mid - 1$)

6. **else return** *binarysearch*($mid + 1, high$)

7. **end if**

算法分析

假设

假定每个三向比较当做一次比较来计算

$$C(n) \leq \begin{cases} 1 & \text{若 } n = 1 \\ 1 + C(\lfloor n/2 \rfloor) & \text{若 } n \geq 2 \end{cases}$$

定理3.2

算法BINARYSEARCHREC在n个元素组成的数组中搜索某个元素所执行的元素比较次数不超过 $\lfloor \log n \rfloor + 1$ 。时间复杂性是 $O(\log n)$ 。

- 由于算法递归深度为 $O(\log n)$ ，并且由于每个递归层次需要 $\Theta(1)$ 的空间，因此，本递归算法所需的空间总量是 $\Theta(\log n)$ 。如果改为迭代形式的算法（算法1.2），仅需要 $\Theta(1)$ 空间。

3.1.2 归并排序（合并排序）

- 第一讲介绍的BOTTOMUPSORT算法：
 - 在每一层，我们有已排序的序列对，同时它们两两被合并而得到较大的排序序列。
 - 沿着树一层一层向上继续这个过程，直到到达根为止，这最后的序列是已经排序好的。

让我们从反向来考虑，用自顶向下取代自底向上

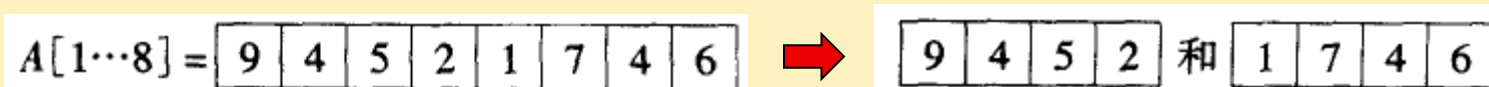
==合并排序

合并排序



基本思想

- 首先，将该数组分成两个平衡的子数组，例：



- 接着，分别对这两个子数组各自进行排序
- 最后，将两个排好序的子数组合并得到所希望的排序数组
- 说明：对每一半子数组可随意使用任何排序算法来排序。如果使用算法自身的思想进行子数组排序，就是著名的算法基于分治策略的合并排序算法 MERGESORT。



算法3.3 MERGESORT

输入: n 个元素的数组 $A[1..n]$ 。

输出: 按非降序排列的数组 $A[1..n]$ 。

1. *mergesort*($A, 1, n$)

过程 *mergesort*($A, low, high$)

1. **if** $low < high$ **then**

2. $mid \leftarrow \lfloor (low + high) / 2 \rfloor$

3. *mergesort*(A, low, mid)

4. *mergesort*($A, mid + 1, high$)

5. *MERGE*($A, low, mid, high$)

6. **end if**

算法执行过程举例

设用算法MERGESORT对下列数组A排序：

$A = \{9, 4, 5, 2, 1, 7, 4, 6\}$.

MERGESORT算法执行过程如下页所示：

- mergesort(A, low, high) 简记为 MS(low, high).
- Merge(A, low, mid, high) 简记为 M(low, mid, high).





$MS(1,8)$

$M(1,4,8)$

9	4	5	2	1	7	4	6
1	2	4	4	5	6	7	9

$MS(1,4)$

$M(1,2,4)$

9	4	5	2
2	4	5	9

$MS(5,8)$

$M(5,6,8)$

1	7	4	6
1	4	6	7

$MS(1,2)$

$M(1,1,2)$

9	4
4	9

$MS(3,4)$

$M(3,3,4)$

5	2
2	5

$MS(5,6)$

$M(5,5,6)$

1	7
1	7

4	6
4	6

$MS(7,8)$

$M(7,7,8)$

$MS(1,1)$

$M(1,1,1)$

9
9

$MS(2,2)$

$M(2,2,2)$

4
4

$MS(3,3)$

$M(3,3,3)$

5
5

$MS(4,4)$

$M(4,4,4)$

2
2

$MS(5,5)$

$M(5,5,5)$

1
1

$MS(6,6)$

$M(6,6,6)$

7
7

$MS(7,7)$

$M(7,7,7)$

4
4

$MS(8,8)$

$M(8,8,8)$

6
6

Success!

- **调用过程**对应于树的前序遍历:访问根、左子树和右子树。
- **计算过程**却对应于树的后序遍历:访问左子树, 右子树, 最后是根。
- 为了实现这个遍历, 用一个**栈**来存储每次调用过程的局部数据。
- 在算法**BOTTOMUPSORT**中, 合并是一层接一层进行的;而在算法**MERGESORT**中, 合并是以后序遍历的方式进行的。两种算法的**区别仅在于合并的次序**, 因此, 当 n 是2的幂时, 由算法**MERGESORT**执行的比较次数和算法**BOTTOMUPSORT**执行的次数相同。

算法分析

- 基本运算: 元素比较。为了简单, 假定 n 是2的幂。
- 设对大小为 n 的数组排序, 元素比较总次数为 $c(n)$
 - 如果 $n=1$, 则算法不执行任何元素的比较, $c(n)=0$ 。
 - 如果 $n \geq 2$, 将问题分解为2个大小为 $n/2$ 的子问题, 分别对两个子数组排序, 一共需要 $2c(n/2)$ 。
 - 合并两个排好序的子数组合并, 合并所需的元素比较次数在 $n/2$ 与 $n-1$ 之间。
- 因此, 算法所需的最小比较次数:

$$c(n) = \begin{cases} 0 & \text{if } n=1 \\ 2c(n/2) + n/2 & \text{if } n \geq 2 \end{cases} \quad \longrightarrow \quad c(n) = \frac{n \log n}{2}$$

算法分析

- 最大比较次数:

$$c(n) = \begin{cases} 0 & \text{if } n = 1 \\ 2c(n/2) + n - 1 & \text{if } n \geq 2 \end{cases}$$

$$\begin{aligned} c(n) &= 2c(n/2) + n - 1 \\ &= 2(2c(n/2^2) + n/2 - 1) + n - 1 \\ &= 2^2 c(n/2^2) + n - 2 + n - 1 \\ &= 2^2 c(n/2^2) + 2n - 2 - 1 \end{aligned}$$

.....

$$\begin{aligned} &= 2^k c(n/2^k) + kn - 2^{k-1} - 2^{k-2} - \dots - 2 - 1 \\ &= 2^k c(1) + kn - \sum_{j=0}^{k-1} 2^j \\ &= 2^k \times 0 + kn - (2^k - 1) \\ &= kn - 2^k + 1 \\ &= n \log n - n + 1 \end{aligned}$$



算法分析

- 如果 n 是任意的正整数(不必是2的幂), 则算法 MERGESORT 执行的元素比较次数如下:

$$c(n) = \begin{cases} 0 & \text{if } n = 1 \\ c(\lfloor n/2 \rfloor) + c(\lceil n/2 \rceil) + bn & \text{if } n \geq 2 \end{cases}$$

$$c(n) = \Theta(n \log n)$$

定理3.3 算法MERGESORT对一个 n 个元素的数组排序所需的时间是 $\Theta(n \log n)$ ，空间是 $\Theta(n)$ （练习）。

主要内容



3.1 引言

3.2 分治算法的简单例

3.3 分治策略的思想

3.4 分治策略的经典应用

3.5 分治算法的优化



分治范式

分治范式通常由以下的步骤组成：

- **划分步骤。**在算法的这个步骤中，输入被分成 $p \geq 1$ 个部分，每个子实例的规模严格小于 n ， n 是原始实例的规模。尽管比2大的其他小的常数很普遍，但 p 的最常用的值是2。 p 也可以具有与 $\log n$ ， n^ϵ 一样高的数量级($0 < \epsilon < 1$)。
- **治理步骤。**如果问题的规模大于某个预定的**阈值** n_0 ，这个步骤由执行 p 个递归调用组成，阈值是由算法的数学分析导出的。
- **组合步骤。**这个步骤是组合 p 个递归调用的解来得到期望的输出值。

分治算法的一般求解过程

分治法通常采用递归算法设计技术，在每一层递归上都有3个步骤：

- ① 分解：将原问题分解为若干个规模较小，相互独立，与原问题形式相同的子问题。
- ② 求解子问题：若子问题规模较小而容易被解决则直接求解，否则递归地求解各个子问题。
- ③ 合并：将各个子问题的解合并为原问题的解。

分治算法的一般形式

- (1) 如果实例I的规模足够“小”，则使用直接的方法求解问题并返回答案，否则，继续做下一步；
- (2) 把实例I分割成p个大小几乎相等的子实例；
- (3) 对每个子实例，递归调用算法，得到p个部分解；
- (4) 组合p个解的结果，得到原问题实例I的解，返回实例I的解。

分治法适用的问题

分治法所能解决的问题一般具有以下几个特征：

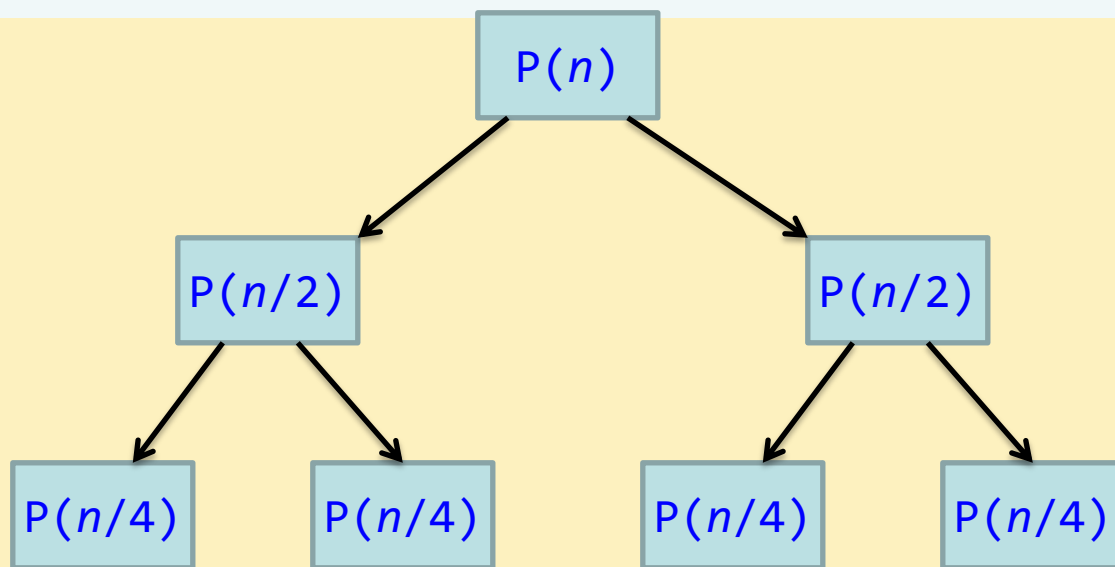
- 该问题的规模缩小到一定的程度就可以容易地解决；
- 该问题可以分解为若干个规模较小的相同问题，即具有最优子结构性质（问题的最优解所包含的子问题的解也是最优的）
- 利用该问题分解出的子问题的解可以合并为该问题的解；
- 该问题所分解出的各个子问题是相互独立的，即子问题之间不包含公共的子问题。

分割原则

- 根据分治法的分割原则，原问题应该分为多少个子问题才较适宜？各个子问题的规模应该怎样才为适当？
- 这些问题很难予以肯定的回答。但人们从大量实践中发现，在用分治法设计算法时，最好使子问题的规模大致相同。换句话说，将一个问题分成大小相等的 p 个子问题的处理方法是行之有效的。 $p=2$ 是最常见的值。
- 当 $p=1$ 时称为**减治法**。例如，BINARYSEARCH REC中 $p=1$ ，输入一部分数据被丢弃。

分割原则

$p=2$ ，称为二分法，如下图所示。使子问题规模大致相等的做法是出自一种平衡子问题的思想，它几乎总是比子问题规模不等的做法要好。



...

...

...

...



分治法的优点

以BOTTOMUPSOT算法和MERGESORT算法对比为例。

- 考虑到因递归调用使用堆栈而需要的额外空间数，以及处理递归调用内在开销所带来的额外时间。从实践的观点看，没有利用赞成使用递归的MERGESORT算法而放弃迭代的BOTTOMUP算法，那么为什么还需要借助MERGESORT 这样的递归算法呢？
- 从理论观点来看，递归算法具有对问题易于叙述、理解和分析的特点。与MERGESORT算法相比， BOTTOMUP需要更多的时间来调试和理解代码的含义。
- 启示算法设计者从递归的框架着手是更好的选择，如有可能，再将算法改进并转化为一个迭代的算法。每个递归算法均可转换为迭代算法，且两者功能完全相同。